

Sprint 11: Accessible HTML Patterns — Full Structured Notes in Simple Words

Sprint 11 Goal

The goal of Sprint 11 is to learn **real accessible HTML patterns**.

In Sprint 10, you learned the basic idea of accessibility:

Every type of user should be able to understand and use your web page.

In Sprint 11, you learn how to write HTML that supports this goal.

By the end of Sprint 11, you should understand:

- Why headings must be logical and descriptive
- Why forms need real labels
- How to make tables easier for screen reader users
- How to write useful alt text for images
- When to use empty alt text
- How linked images should be described
- Why link text must explain the destination
- How captions, subtitles, and transcripts help media accessibility
- Why keyboard navigation matters
- How natural tab order works
- What `tabindex` does
- Why positive `tabindex` values are dangerous
- Why visible focus indicators are important
- What keyboard traps are
- How to combine these patterns in one accessible HTML page

1. Where You Are Now

You have already completed many important HTML foundations.

Sprint	Main Topic You Learned
Sprint 1	HTML elements, tags, nesting, attributes, void elements
Sprint 2	Full HTML boilerplate, <code>head</code> , metadata, <code>body</code>
Sprint 3	Text meaning with <code>em</code> , <code>strong</code> , <code>abbr</code> , <code>time</code> , <code>code</code> , <code>pre</code>

Sprint	Main Topic You Learned
Sprint 4	Links, paths, <code>mailto:</code> , <code>tel:</code> , navigation
Sprint 5	Images, audio, video, SVG, iframe, media responsibility
Sprint 6	Semantic layout: <code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>article</code> , <code>footer</code>
Sprint 7	Form basics, labels, inputs, placeholders
Sprint 8	Buttons, validation, disabled, readonly, focus states
Sprint 9	Tables, captions, table structure, debugging, validators
Sprint 10	Accessibility foundations, WCAG, POUR, assistive technology

Now Sprint 11 asks a better question:

Can real users, including screen reader users and keyboard users, understand and use my HTML page?

Simple idea:

Sprint 10 explained why accessibility matters. Sprint 11 teaches practical HTML patterns that make pages more accessible.

2. What “Accessible HTML Patterns” Means

An accessible HTML pattern is a way of writing HTML that helps more users use the page.

For example, this is a weak form pattern:

```
<input type="email" placeholder="Email Address">
```

It may look okay, but it has no real label.

This is a better accessible pattern:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Why better?

- The label explains the field.
- The label is connected to the input.

- Screen readers can announce the field clearly.
- Clicking the label can focus the input.

Simple rule:

Accessible HTML is not only about using valid tags. It is about using tags in a way that helps real people.

3. Accessible Headings

3.1 What Headings Do

Headings are not just big text.

Headings create the structure of the page.

They help users understand:

- What the page is about
- What the main sections are
- What smaller subsections belong inside bigger sections
- Where they are on the page

Screen reader users can often open a list of headings and jump to the section they want.

So headings work like a **page map**.

3.2 Heading Levels

HTML has six heading levels:

```
<h1>Main page topic</h1>  
<h2>Main section</h2>  
<h3>Subsection</h3>  
<h4>Smaller subsection</h4>  
<h5>Lower-level heading</h5>  
<h6>Lowest-level heading</h6>
```

Simple meaning:

Heading	Meaning
<code>h1</code>	Main page topic
<code>h2</code>	Major section
<code>h3</code>	Subsection inside an <code>h2</code>
<code>h4</code>	Subsection inside an <code>h3</code>
<code>h5</code>	Subsection inside an <code>h4</code>
<code>h6</code>	Subsection inside an <code>h5</code>

3.3 Use One Clear `h1`

For normal beginner pages, use one main `h1`.

Example:

```
<h1>HTML Accessibility Guide</h1>
```

This tells the user:

This page is mainly about HTML accessibility.

Then use `h2` for main sections.

```
<h1>HTML Accessibility Guide</h1>

<h2>Headings</h2>
<h2>Forms</h2>
<h2>Tables</h2>
<h2>Media</h2>
```

Simple rule:

Use `h1` for the main page title. Use `h2` for major sections.

3.4 Do Not Skip Heading Levels

Bad:

```
<h1>HTML Accessibility Guide</h1>
<h3>Forms</h3>
```

Why bad?

The page jumps from `h1` to `h3`.

The `h2` level is missing.

This can make the page outline confusing.

Better:

```
<h1>HTML Accessibility Guide</h1>
<h2>Forms</h2>
<h3>Email Field</h3>
```

This structure is clearer:

```
HTML Accessibility Guide
  Forms
    Email Field
```

Simple rule:

Do not jump from `h1` to `h3`. Move down one level at a time.

3.5 Good Heading Structure Example

```
<h1>Beginner HTML Course</h1>

<h2>Introduction</h2>
<p>This course teaches HTML from the beginning.</p>

<h2>Lessons</h2>

<h3>HTML Elements</h3>
<p>Elements structure the page.</p>

<h3>HTML Forms</h3>
<p>Forms collect user input.</p>
```

```
<h2>Practice Projects</h2>
<p>You will build small pages.</p>
```

Why this is good:

- There is one clear `h1`.
- Main sections use `h2`.
- Smaller sections inside Lessons use `h3`.
- The order is logical.

3.6 Bad Heading Structure Example

```
<h1>Beginner HTML Course</h1>
<h4>Introduction</h4>
<h2>Lessons</h2>
<h5>HTML Elements</h5>
```

Why this is bad:

- Heading levels jump around.
- The structure is not clear.
- It may confuse screen reader navigation.
- It may confuse other developers reading the code.

3.7 Do Not Use Headings Only for Size

Bad thinking:

I want this text to look big, so I will use `h1`.

Better thinking:

This is the main page title, so I will use `h1`.

Bad:

```
<h1>Product Name</h1>
<h1>Price</h1>
<h1>Reviews</h1>
```

Better:

```
<h1>Product Name</h1>
<h2>Price</h2>
<h2>Reviews</h2>
```

Simple rule:

Use headings for structure, not for visual size. Use CSS later for appearance.

4. Accessible Form Labels

4.1 Why Labels Matter

A label tells the user what an input is for.

Bad:

```
<input type="text">
```

Problem:

The user does not know what to type.

Better:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name">
```

Now the user understands:

This field is for my full name.

Labels help:

- Sighted users
- Screen reader users
- Keyboard users
- People with motor difficulties
- People who need clear instructions

Simple rule:

Every input should have a clear label.

4.2 Explicit Label Association

Explicit label association uses:

- `for` on the `label`
- `id` on the `input`

The value of `for` must match the value of `id`.

Example:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Here:

```
label for="email"
input id="email"
```

They match.

So the label is connected to the input.

Simple meaning:

This label belongs to the input with `id="email"`.

4.3 Explicit Labels Are a Strong Beginner Pattern

This pattern is easy to read:

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

A beginner can clearly see the connection.

Recommended beginner pattern:

```
<label for="field-id">Field Name:</label>
<input type="text" id="field-id" name="field-id">
```

Simple rule:

For explicit labels, `label for` must match `input id`.

4.4 Placeholder Is Not a Label

A placeholder is a short hint inside an input.

Example:

```
<input type="email" placeholder="example@email.com">
```

But a placeholder must not replace a real label.

Bad:

```
<input type="email" placeholder="Email Address">
```

Why bad?

- There is no real label.
- The placeholder disappears when the user types.
- Some users may forget what the field is for.
- Screen reader support may be weaker.
- It is less accessible.

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Simple rule:

Label = field name. Placeholder = temporary hint.

4.5 Good Form Example

```
<form action="/contact">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name" required>

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" placeholder="example@email.com"
required>

  <label for="message">Message:</label>
  <input type="text" id="message" name="message" minlength="10" maxlength="200">

  <button type="submit">Send Message</button>
</form>
```

Why this is good:

- Every input has a label.
- Every label is connected to the correct input.
- The button has a clear type.
- Required fields use `required`.
- Placeholder is only used as a hint.

4.6 Bad Form Example

```
<form>
  <input type="text" placeholder="Full Name">
  <input type="email" placeholder="Email">
  <button>Send</button>
</form>
```

Problems:

- No `action` attribute.
- Inputs have no real labels.
- Placeholders are being used as labels.
- Button has no explicit `type`.

Better:

```
<form action="/contact">
  <label for="full-name">Full Name:</label>
```

```
<input type="text" id="full-name" name="full-name">

<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">

<button type="submit">Send Message</button>
</form>
```

5. Accessible Tables

5.1 Tables Must Be Used for Real Data

A table should be used for data that naturally fits into rows and columns.

Good table content:

Course	Duration	Level
HTML	10 hours	Beginner
CSS	15 hours	Beginner

This is good because the data has clear rows and columns.

Bad use:

```
<table>
  <tr>
    <td>Sidebar</td>
    <td>Main content</td>
  </tr>
</table>
```

Why bad?

This table is being used for layout, not real data.

Better semantic structure:

```
<main>
  <aside>
    <h2>Related Links</h2>
```

```
<p>Extra information goes here.</p>
</aside>

<section>
  <h2>Main Content</h2>
  <p>Main page content goes here.</p>
</section>
</main>
```

Simple rule:

Use tables for data. Do not use tables for page layout.

5.2 Use a `caption`

The `caption` element gives the table a title or explanation.

Example:

```
<table>
  <caption>Course Duration Comparison</caption>
</table>
```

The caption helps users understand what the table is about.

It is especially useful for screen reader users.

Correct placement:

```
<table>
  <caption>Course Duration Comparison</caption>
  ...
</table>
```

The `caption` should come immediately after the opening `<table>` tag.

Simple rule:

Use `caption` to explain the table purpose early.

5.3 Use `th` for Header Cells

Use `th` when a cell is a heading or label.

Example:

```
<th>Course</th>
<th>Duration</th>
<th>Level</th>
```

Use `td` for normal data.

Example:

```
<td>HTML</td>
<td>10 hours</td>
<td>Beginner</td>
```

Simple memory rule:

```
th = table heading
td = table data
```

5.4 The `scope` Attribute

The `scope` attribute tells assistive technology what a header cell applies to.

Two common values are:

Value	Meaning
<code>scope="col"</code>	This header applies to a column
<code>scope="row"</code>	This header applies to a row

Column header example:

```
<th scope="col">Course</th>
```

This means:

Course is the heading for this column.

Row header example:

```
<th scope="row">HTML</th>
```

This means:

HTML is the heading for this row.

Simple rule:

Use `scope` to make the relationship between headers and data clearer.

5.5 Complete Accessible Table Example

```
<table>
  <caption>Course Duration Comparison</caption>

  <thead>
    <tr>
      <th scope="col">Course</th>
      <th scope="col">Duration</th>
      <th scope="col">Level</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <th scope="row">HTML</th>
      <td>10 hours</td>
      <td>Beginner</td>
    </tr>

    <tr>
      <th scope="row">CSS</th>
      <td>15 hours</td>
      <td>Beginner</td>
    </tr>

    <tr>
      <th scope="row">JavaScript</th>
      <td>25 hours</td>
```

```
<td>Intermediate</td>
</tr>
</tbody>
</table>
```

Why this table is accessible:

- It uses a clear `caption`.
- Column headings use `scope="col"`.
- Row headings use `scope="row"`.
- The structure is simple.
- It avoids unnecessary complex merged cells.

5.6 Keep Tables Simple

Complicated tables can be hard for everyone.

Avoid too much:

- `colspan`
- `rowspan`
- Many nested headers
- Too many columns
- Fixed widths that force horizontal scrolling

Simple tables are easier for:

- Screen reader users
- Mobile users
- Low-vision users
- Developers maintaining the code

Simple rule:

Keep tables as simple and flat as possible.

5.7 Avoid Fixed Widths and Heights

A table with fixed width can cause problems on small screens or zoomed pages.

Weak idea:

```
<table width="1200">
```

Why weak?

- It may force horizontal scrolling.
- It may not fit mobile screens.
- It can be difficult for zoom users.

Better idea:

Use clean HTML structure first. Handle layout carefully with CSS later.

Simple rule:

Do not make tables harder to use by forcing fixed sizes in HTML.

6. Accessible Images and Alt Text

6.1 What Alt Text Does

The `alt` attribute gives a text alternative for an image.

Example:

```

```

Alt text helps when:

- The image does not load
- A screen reader user cannot see the image
- The page is used in low-bandwidth conditions
- The image contains meaningful information

Simple rule:

Alt text should explain the useful meaning of the image.

6.2 Alt Text Depends on Image Purpose

You should not write alt text only by describing pixels.

Ask this question:

Why is this image on the page?

The same image may need different alt text in different situations.

Example image: a photo of a red shoe.

On a product page:

```

```

In a decorative background area:

```

```

In a link to buy the shoe:

```
<a href="buy-red-shoe.html">  
    
</a>
```

Simple rule:

The best alt text depends on the job of the image.

6.3 Meaningful Images Need Useful Alt Text

A meaningful image gives useful information.

Example:

```

```

This is better than:

```

```

Why?

`alt="image"` does not explain anything useful.

Good alt text should be:

- Short enough to be readable
- Specific enough to explain meaning
- Focused on what matters in context

Simple rule:

Do not write `alt="image"`, `alt="photo"`, or `alt="picture"` as your full alt text.

6.4 Decorative Images Use Empty Alt Text

A decorative image is an image that does not add useful information.

Examples:

- Border pattern
- Background decoration
- Decorative icon repeated beside text
- Divider image

For decorative images, use empty alt text:

```

```

Why?

A screen reader can skip it.

If you omit the `alt` attribute completely, the screen reader may read the file name.

Bad:

```

```

Possible screen reader result:

```
border-pattern dot png
```

That is not useful.

Better:

```

```

Simple rule:

Decorative image? Use `alt=""`.

6.5 Linked Image Alt Text

When an image is inside a link, the alt text should describe the link action or destination.

Example:

```
<a href="next-page.html">
  
</a>
```

This is better than:

```
<a href="next-page.html">
  
</a>
```

Why?

The user does not only need to know that the image is an arrow.

The user needs to know what happens when they activate the link.

Simple rule:

If an image is a link, alt text should describe the action or destination.

6.6 Image with Caption

Sometimes an image needs a visible caption.

Use `figure` and `figcaption`.

Example:

```
<figure>
  
  <figcaption>A basic semantic HTML page layout.</figcaption>
</figure>
```

Why this is good:

- The image has alt text.
- The visible caption explains the figure.
- The structure is semantic.

Simple rule:

Use `figure` and `figcaption` when media needs a visible caption.

7. Accessible Link Text

7.1 Why Link Text Matters

The text inside a link is what the user activates.

Example:

```
<a href="resources.html">View HTML learning resources</a>
```

Good link text tells the user where the link goes.

Screen reader users may browse a list of links on the page.

If every link says “click here”, that list becomes confusing.

Bad:

```
<a href="resources.html">Click here</a>
<a href="pricing.html">Click here</a>
<a href="contact.html">Click here</a>
```

A screen reader link list may sound like:

```
Click here
Click here
Click here
```

That is not helpful.

Better:

```
<a href="resources.html">View HTML learning resources</a>
<a href="pricing.html">View course pricing details</a>
<a href="contact.html">Contact the course support team</a>
```

Now the destination is clear.

7.2 Avoid Vague Link Text

Avoid link text like:

```
Click here
Read more
Details
More info
Learn more
Here
This page
```

These may be unclear without surrounding text.

Better examples:

```
<a href="forms-guide.html">Read the HTML forms guide</a>
<a href="timetable.pdf">Download the course timetable</a>
```

```
<a href="accessibility-checklist.html">View the accessibility checklist</a>  
<a href="support.html">Contact the support team</a>
```

Simple rule:

Link text should describe the destination or action.

7.3 Good Link Text Examples

Weak Link Text	Better Link Text
Click here	View HTML accessibility resources
More info	View course pricing details
Read more	Read the full article about semantic HTML
Details	View registration details
Download	Download the HTML practice worksheet

7.4 Same Link Text Should Usually Go to Same Destination

If two links have the same text, users may expect them to go to the same place.

Weak:

```
<a href="html.html">Read more</a>  
<a href="css.html">Read more</a>
```

Better:

```
<a href="html.html">Read more about HTML</a>  
<a href="css.html">Read more about CSS</a>
```

Simple rule:

Repeated link text should not create confusion.

8. Accessible Media: Captions, Subtitles, and Transcripts

8.1 Why Media Accessibility Matters

Media includes:

- Audio
- Video
- Embedded lessons
- Interviews
- Tutorials
- Presentations

Not every user can hear audio.

Not every user can watch video easily.

Some users may be:

- Deaf
- Hard of hearing
- In a noisy place
- In a quiet place without headphones
- Using a slow connection
- Trying to search text content quickly

So media should have text alternatives.

Simple rule:

If audio or video gives important information, provide a text alternative.

8.2 Captions

Captions are text shown with video.

They usually include:

- Spoken words
- Important sounds
- Speaker information when needed

Example:

```
<video controls width="400" poster="lesson-poster.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions.vtt" kind="captions" srclang="en" label="English">
  Your browser does not support the video element.
</video>
```

Important parts:

Attribute	Meaning
<code>src</code>	Location of the caption file
<code>kind="captions"</code>	The track is for captions
<code>srclang="en"</code>	The caption language is English
<code>label="English"</code>	Human-readable name shown to users

Simple rule:

Use captions when video has speech or important sound.

8.3 Subtitles

Subtitles are usually for translating spoken language.

Example:

```
<track src="subtitles-si.vtt" kind="subtitles" srclang="si" label="Sinhala">
```

This means:

Use this file as Sinhala subtitles.

Simple difference:

Type	Main Purpose
Captions	Help users understand speech and important sounds
Subtitles	Translate spoken language into another language

Beginner rule:

Captions are mainly for accessibility. Subtitles are often for language translation.

8.4 Transcripts

A transcript is a written version of audio or video content.

Example:

```
<section>
  <h2>Video Transcript</h2>
  <p>In this lesson, we explain how to write accessible headings, labels, and
links.</p>
</section>
```

Transcripts help:

- Deaf users
- Hard-of-hearing users
- Users who prefer reading
- Users with slow internet
- Users who want to search the content
- Users who cannot play audio at the moment

Simple rule:

A transcript gives users another way to get the media content.

8.5 Use Controls on Audio and Video

Users need control over media playback.

Good:

```
<audio controls>
  <source src="lesson.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Good:

```
<video controls width="400">
  <source src="lesson.mp4" type="video/mp4">
  Your browser does not support the video element.
</video>
```

Why controls matter:

Users can:

- Play
- Pause
- Change volume
- Replay
- Stop when needed

Weak:

```
<audio src="lesson.mp3"></audio>
```

Problem:

There are no visible controls.

Simple rule:

Audio and video should usually have `controls`.

8.6 Be Careful with Autoplay

Autoplay can be harmful.

Problems:

- It may annoy users.
- It may surprise screen reader users.
- It may use data unexpectedly.
- It may create noise in public places.
- Some browsers block autoplay.

Simple rule:

Prefer user-controlled media playback.

9. Keyboard Navigation

9.1 Why Keyboard Navigation Matters

Not all users use a mouse.

Some users use:

- Keyboard only
- Switch devices
- Voice recognition software
- Screen readers
- Alternative pointing devices
- Special accessibility hardware

Keyboard users often use:

Key	Common Use
<code>Tab</code>	Move to next focusable item
<code>Shift + Tab</code>	Move to previous focusable item
<code>Enter</code>	Activate links and buttons
<code>Space</code>	Activate buttons and some controls
Arrow keys	Move inside some controls

Simple rule:

A page should work without a mouse.

9.2 Use Real Links for Navigation

Good:

```
<a href="contact.html">Contact us</a>
```

Why good?

A real link can be:

- Focused with keyboard
- Opened with keyboard

- Understood by screen readers
- Recognized as navigation

Bad:

```
<div onclick="goToContact()">Contact us</div>
```

Why bad?

A `div` is not naturally a link.

It may not work correctly with keyboard or assistive technology.

Simple rule:

Use `<a>` when the user goes somewhere.

9.3 Use Real Buttons for Actions

Good:

```
<button type="submit">Submit Application</button>
```

Why good?

A real button can be:

- Focused with keyboard
- Activated by keyboard
- Understood as a button
- Announced correctly by screen readers

Bad:

```
<div onclick="submitForm()">Submit</div>
```

Why bad?

A `div` is not naturally a button.

Simple rule:

Use `<button>` when the user performs an action.

9.4 Link vs Button

Beginners often confuse links and buttons.

Use This	When
<code><a></code>	The user goes to another page, section, file, email, or phone link
<code><button></code>	The user performs an action on the current page or submits a form

Examples:

```
<a href="courses.html">View courses</a>
<button type="submit">Submit form</button>
```

Simple rule:

Link = navigation. Button = action.

9.5 Natural Tab Order

Natural tab order means the keyboard focus follows the order of the HTML.

Example:

```
<a href="index.html">Home</a>
<a href="courses.html">Courses</a>
<a href="contact.html">Contact</a>
<button type="submit">Submit</button>
```

When the user presses `Tab`, focus moves in this order:

1. Home
2. Courses
3. Contact
4. Submit

This is natural and predictable.

Simple rule:

Write HTML in a logical order so keyboard focus follows a logical order.

10. tabindex

10.1 What tabindex Does

The `tabindex` attribute controls keyboard focus behavior.

Common values:

Value	Meaning
<code>tabindex="0"</code>	Adds the element to the natural tab order
<code>tabindex="-1"</code>	Allows focus by script, but not by normal Tab navigation
<code>tabindex="1"</code> or higher	Creates a custom tab order; usually bad

Beginner warning:

Most of the time, you should not need `tabindex` if you use real HTML elements correctly.

10.2 tabindex="0"

`tabindex="0"` adds an element to the natural keyboard order.

Example:

```
<div tabindex="0">Focusable custom area</div>
```

This means the user can reach this `div` by pressing `Tab`.

But be careful.

A focusable `div` still does not become a real button or real link.

Weak:

```
<div tabindex="0">Submit</div>
```

Better:

```
<button type="submit">Submit</button>
```

Simple rule:

Do not use `tabindex="0"` to fake real controls. Use real controls first.

10.3 `tabindex="-1"`

`tabindex="-1"` means:

- The element is not reached by normal Tab navigation.
- The element can still receive focus by script or special behavior.

Example:

```
<section tabindex="-1">
  <h2>Error Summary</h2>
  <p>Please fix the form errors below.</p>
</section>
```

Beginner idea:

You do not need to use this often in basic HTML.

For Sprint 11, mainly recognize what it means.

Simple rule:

`tabindex="-1"` removes an element from normal tab order but can still allow focus in special cases.

10.4 Avoid Positive `tabindex`

Positive `tabindex` values create custom keyboard order.

Bad:

```
<button tabindex="5">Submit</button>
<a href="home.html" tabindex="1">Home</a>
<input type="email" tabindex="3">
```

Why bad?

The visual order and keyboard order can become different.

This confuses keyboard users.

Better:

```
<a href="home.html">Home</a>
<input type="email" id="email" name="email">
<button type="submit">Submit</button>
```

Let the browser follow the natural HTML order.

Simple rule:

Avoid positive `tabindex` values.

11. Visible Focus Indicators

11.1 What Focus Means

Focus means the current selected interactive element.

A focused element might be:

- A link
- A button
- An input
- A select menu
- A textarea

Keyboard users need to see which element is focused.

When a user presses `Tab`, the focus should be visible.

Simple rule:

If users cannot see where focus is, they may get lost.

11.2 Do Not Remove Focus Outline Carelessly

Bad CSS:

```
button:focus {  
  outline: none;  
}
```

Why bad?

The focus indicator disappears.

A keyboard user may not know where they are.

Better idea:

```
button:focus {  
  outline: 3px solid black;  
}
```

You do not need to master CSS in this HTML sprint.

But remember this important accessibility rule:

Do not remove focus indicators unless you replace them with a clear visible focus style.

11.3 Focus Indicator Contrast

A focus indicator should be visible enough against the background.

If the focus outline is too faint, users may miss it.

Bad idea:

```
button:focus {  
  outline: 1px solid lightgray;  
}
```

Problem:

It may be too hard to see.

Better idea:

```
button:focus {  
  outline: 3px solid black;  
}
```

Simple rule:

Focus indicators should be clear and easy to see.

12. Keyboard Traps

12.1 What Is a Keyboard Trap?

A keyboard trap happens when a keyboard user can enter a part of the page but cannot leave it.

Example situation:

A popup opens.

The user presses Tab repeatedly.

Focus stays trapped inside one area and the user cannot reach the close button or return to the page.

This is a serious accessibility problem.

Simple meaning:

A keyboard trap blocks the user from moving freely through the page.

12.2 Why Keyboard Traps Are Bad

Keyboard traps are bad because users may be unable to:

- Close a popup
- Continue filling a form
- Reach navigation

- Submit information
- Return to main content
- Escape from a component

A mouse user might click somewhere else.

But a keyboard-only user may be stuck.

Simple rule:

Keyboard users must be able to enter, use, and leave every interactive area.

12.3 How to Think About Keyboard Testing

When testing your page, try this:

1. Put your mouse away.
2. Press `Tab` to move forward.
3. Press `Shift + Tab` to move backward.
4. Press `Enter` or `Space` to activate controls.
5. Check whether you can reach every link, button, and input.
6. Check whether you can leave every section or popup.
7. Check whether the focus indicator is visible.

Simple rule:

If you cannot use the page with only the keyboard, some users may not be able to use it either.

13. Access Keys

13.1 What `accesskey` Does

The `accesskey` attribute can create keyboard shortcuts for elements.

Example:

```
<a href="home.html" accesskey="h">Home</a>
```

This may allow a keyboard shortcut for the Home link.

But `accesskey` can cause problems.

13.2 Why `accesskey` Can Be Risky

Different browsers and operating systems use different shortcut combinations.

Problems:

- Shortcuts may conflict with browser shortcuts.
- Shortcuts may conflict with assistive technology shortcuts.
- Users may not know the shortcut exists.
- The same key may behave differently in different browsers.

Simple beginner rule:

Be careful with `accesskey`. Do not use it unless there is a strong reason and proper testing.

14. Putting Sprint 11 Patterns Together

This example combines accessible headings, labels, tables, images, links, media, and keyboard-friendly controls.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Accessible HTML Patterns</title>
</head>
<body>
  <header>
    <h1>Accessible HTML Patterns</h1>

    <nav>
      <a href="index.html">Go to the home page</a>
      <a href="forms.html">Read the accessible forms guide</a>
      <a href="tables.html">View accessible table examples</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Accessible Form Example</h2>

      <form action="/contact">
```

```

<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name" required>

<label for="email">Email Address:</label>
<input type="email" id="email" name="email"
placeholder="example@email.com" required>

<button type="submit">Send Contact Form</button>
</form>
</section>

<section>
<h2>Accessible Table Example</h2>

<table>
<caption>Beginner Web Course Comparison</caption>

<thead>
<tr>
<th scope="col">Course</th>
<th scope="col">Duration</th>
<th scope="col">Level</th>
</tr>
</thead>

<tbody>
<tr>
<th scope="row">HTML</th>
<td>10 hours</td>
<td>Beginner</td>
</tr>

<tr>
<th scope="row">CSS</th>
<td>15 hours</td>
<td>Beginner</td>
</tr>
</tbody>
</table>
</section>

<section>
<h2>Accessible Image Example</h2>

<figure>

<figcaption>A simple semantic HTML page layout.</figcaption>

```

```

    </figure>

    
</section>

<section>
  <h2>Accessible Video Example</h2>

  <video controls width="400" poster="lesson-poster.jpg">
    <source src="lesson.mp4" type="video/mp4">
    <track src="captions.vtt" kind="captions" srclang="en" label="English">
    Your browser does not support the video element.
  </video>

  <section>
    <h3>Video Transcript</h3>
    <p>This video explains beginner accessible HTML patterns, including
labels, headings, tables, and alt text.</p>
  </section>
</section>
</main>

<footer>
  <p>&copy; 2026 Beginner HTML Course</p>
</footer>
</body>
</html>

```

15. Common Sprint 11 Mistakes

Mistake 1: Using headings for size

Bad:

```

<h1>Page Title</h1>
<h4>About Us</h4>

```

Better:

```

<h1>Page Title</h1>
<h2>About Us</h2>

```

Mistake 2: Using placeholder instead of label

Bad:

```
<input type="email" placeholder="Email Address">
```

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Mistake 3: Missing table caption

Bad:

```
<table>
  <tr>
    <th>Course</th>
    <th>Duration</th>
  </tr>
</table>
```

Better:

```
<table>
  <caption>Course Duration Comparison</caption>
  <tr>
    <th scope="col">Course</th>
    <th scope="col">Duration</th>
  </tr>
</table>
```

Mistake 4: Weak alt text

Bad:

```

```

Better:

```

```

Mistake 5: Missing empty alt for decorative image

Bad:

```

```

Better:

```

```

Mistake 6: Vague link text

Bad:

```
<a href="accessibility.html">Click here</a>
```

Better:

```
<a href="accessibility.html">Read the accessibility guide</a>
```

Mistake 7: Fake button with `div`

Bad:

```
<div onclick="submitForm()">Submit</div>
```

Better:


```
<button type="submit">Submit Form</button>
```

Mistake 8: Positive `tabindex`

Bad:

```
<a href="home.html" tabindex="3">Home</a>
<button tabindex="1">Submit</button>
```

Better:

```
<a href="home.html">Home</a>
<button type="submit">Submit</button>
```

Mistake 9: Removing focus outline

Bad:

```
button:focus {
  outline: none;
}
```

Better:

```
button:focus {
  outline: 3px solid black;
}
```

16. Sprint 11 Quick Checklist

Before finishing an HTML page, ask these questions:

Headings

- Does the page have one clear main `h1`?

- Are `h2` elements used for main sections?
- Are `h3` elements used only inside `h2` sections?
- Did I avoid skipping heading levels?
- Did I avoid using headings only for visual size?

Forms

- Does every input have a real label?
- Does each `label for` match the correct `input id`?
- Did I avoid using placeholder as the only label?
- Are buttons real `button` elements?
- Did I write button `type` clearly?

Tables

- Is the table used only for real data?
- Does the table have a `caption`?
- Are header cells marked with `th`?
- Did I use `scope="col"` for column headers?
- Did I use `scope="row"` where row headers exist?
- Is the table simple and not overcomplicated?

Images

- Does every meaningful image have useful alt text?
- Do decorative images use `alt=""`?
- Does linked image alt text describe the action or destination?
- Did I avoid weak alt text like `image` or `photo`?

Links

- Does every link text explain the destination or action?
- Did I avoid repeated vague links like "click here"?
- Can users understand each link out of context?

Media

- Do videos with speech have captions or subtitles?
- Is there a transcript when useful?
- Do audio and video elements have controls?
- Did I avoid unnecessary autoplay?

Keyboard Access

- Can I use the page with only the keyboard?
- Does the focus order follow the visual/content order?

- Are focus indicators visible?
 - Did I avoid positive `tabindex`?
 - Did I avoid keyboard traps?
-

17. Sprint 11 Revision Questions

Try to answer these without looking at the notes.

1. Why are headings important for accessibility?
 2. Why should you avoid jumping from `h1` to `h3`?
 3. What is the difference between a label and a placeholder?
 4. How do you connect a `label` to an `input` explicitly?
 5. Why should every form input have a real label?
 6. What does a table `caption` do?
 7. What does `scope="col"` mean?
 8. What does `scope="row"` mean?
 9. When should you use empty alt text?
 10. Why is `alt="image"` weak alt text?
 11. What should linked image alt text describe?
 12. Why is "click here" weak link text?
 13. What is the purpose of captions in video?
 14. What is a transcript?
 15. Why should audio and video usually have controls?
 16. Why should you use a real button instead of a clickable `div`?
 17. What is natural tab order?
 18. What does `tabindex="0"` do?
 19. What does `tabindex="-1"` do?
 20. Why should positive `tabindex` values usually be avoided?
 21. Why are visible focus indicators important?
 22. What is a keyboard trap?
-

18. Mandatory Practice Coding Block

Warm-Up Drills

Write these small code patterns from memory.

Drill 1: Correct heading order

Create:

- One `h1`

- Two `h2` headings
- One `h3` inside the second `h2` section

Drill 2: Accessible input

Create an email input with:

- `label`
- matching `for` and `id`
- `name`
- placeholder as a hint only

Drill 3: Accessible table

Create a table with:

- `caption`
- `thead`
- `tbody`
- column headers using `scope="col"`
- one row header using `scope="row"`

Drill 4: Image alt text

Write:

- One meaningful image with useful alt text
- One decorative image with `alt=""`
- One linked image where the alt describes the action

Drill 5: Descriptive links

Write three links without using:

- Click here
- More info
- Read more

Guided Coding Task

Create a file named:

```
sprint-11-accessible-patterns.html
```

Build a page with:

1. Full HTML boilerplate
2. One `h1`
3. At least three `h2` sections
4. A navigation area with descriptive link text
5. One accessible form with real labels
6. One accessible table with `caption` and `scope`
7. One meaningful image with useful alt text
8. One decorative image with `alt=""`
9. One linked image with action-based alt text
10. One video with a `track` element
11. One transcript section
12. A real submit button
13. Logical HTML order for natural keyboard navigation

19. Debug This Code

Find the accessibility problems:

```
<h1>Course Page</h1>
<h3>Contact Form</h3>

<form>
  <input type="text" placeholder="Full Name">
  <input type="email" placeholder="Email">
  <div onclick="submitForm()">Submit</div>
</form>

<table>
  <tr>
    <td>Course</td>
    <td>Duration</td>
  </tr>
  <tr>
    <td>HTML</td>
    <td>10 hours</td>
  </tr>
</table>



<a href="resources.html">Click here</a>
```

```
<video src="lesson.mp4"></video>
```

Problems:

1. The page jumps from `h1` to `h3`.
2. The form has no labels.
3. Placeholders are used as labels.
4. A clickable `div` is used instead of a real button.
5. The table has no `caption`.
6. Table headings are written as `td` instead of `th`.
7. The image has no `alt` attribute.
8. Link text is vague.
9. The video has no `controls`.
10. The video has no captions or transcript.

Better version:

```
<h1>Course Page</h1>
<h2>Contact Form</h2>

<form action="/contact">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name">

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" placeholder="example@email.com">

  <button type="submit">Send Contact Form</button>
</form>

<table>
  <caption>Course Duration Table</caption>
  <thead>
    <tr>
      <th scope="col">Course</th>
      <th scope="col">Duration</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">HTML</th>
      <td>10 hours</td>
    </tr>
  </tbody>
</table>
```

```


<a href="resources.html">View HTML learning resources</a>

<video controls width="400">
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions.vtt" kind="captions" srclang="en" label="English">
  Your browser does not support the video element.
</video>
```

20. Build Something Small

Build a small page called:

Accessible Course Information Page

Your page should include:

- A page header
- A navigation menu
- A main content area
- A course comparison table
- A registration form
- A course image
- A course video
- A transcript section
- A footer

Accessibility requirements:

- Use clear heading hierarchy.
- Use descriptive link text.
- Use labels for all inputs.
- Use table caption and scope.
- Use meaningful alt text for meaningful images.
- Use empty alt text for decorative images.
- Use video captions.
- Use a transcript.
- Use real links and buttons.
- Avoid positive `tabindex`.

21. Challenge Task

Improve the page further:

1. Add a `figure` and `figcaption` for the course image.
 2. Add a row header in the table using `scope="row"`.
 3. Add a second caption track in another language.
 4. Add a short instruction before the form.
 5. Test the page using only the keyboard.
 6. Check whether the tab order feels natural.
 7. Check whether every link makes sense out of context.
-

22. Sprint 11 Final Memory Rules

Remember these rules before moving to Sprint 12:

- Headings are a page map.
 - Use headings for structure, not size.
 - Use one clear `h1` for a normal beginner page.
 - Do not skip heading levels.
 - Every input needs a real label.
 - `label for` must match `input id`.
 - Placeholder is not a label.
 - Tables need clear structure.
 - Use tables only for tabular data.
 - Use `caption` to explain the table.
 - Use `th` for header cells.
 - Use `scope="col"` for column headers.
 - Use `scope="row"` for row headers.
 - Keep tables simple.
 - Meaningful images need useful alt text.
 - Decorative images need `alt=""`.
 - Linked image alt text describes the action or destination.
 - Link text should describe where the link goes.
 - Avoid vague link text like "click here".
 - Videos with speech need captions or transcripts.
 - Audio and video should usually have controls.
 - Use real links for navigation.
 - Use real buttons for actions.
 - Natural tab order is usually best.
 - Avoid positive `tabindex`.
 - Visible focus indicators are important.
 - Avoid keyboard traps.
-

23. Sprint 11 Definition of Done

You are ready to move to Sprint 12 when you can:

- Explain why headings matter for screen reader users
- Write a correct heading structure from memory
- Create a form input with a connected label
- Explain why placeholder is not a label
- Build an accessible table with `caption`, `th`, and `scope`
- Write useful alt text for meaningful images
- Use `alt=""` for decorative images
- Write action-based alt text for linked images
- Write descriptive link text
- Add a `track` element to a video
- Explain the purpose of captions and transcripts
- Use real links and buttons instead of clickable `div` elements
- Explain natural tab order
- Explain why positive `tabindex` is dangerous
- Explain why focus indicators must be visible
- Identify a possible keyboard trap
- Build a small accessible HTML page using Sprint 11 patterns

24. Simple Final Summary

Sprint 11 is about making your HTML usable by more people.

The main idea is:

Do not only ask, "Does my HTML work?" Ask, "Can different users understand and use it?"

Good accessible HTML uses:

- Logical headings
- Real labels
- Clear tables
- Useful alt text
- Descriptive links
- Captions and transcripts
- Real buttons and links
- Natural keyboard navigation
- Visible focus indicators

This sprint prepares you for Sprint 12, where you will learn WAI-ARIA essentials and build your final HTML capstone.